SPRINT CYCLE III · DATABASE IMPLEMENTATION

Rattler Resource Rescue System

The Sprint II design, built and running in MySQL 8.0.

Lloyd Lewis · Ondrea Wagner · Naser Halloum · Isaiah Miller (Scrum Master)



1 · DATABASE TECHNOLOGY

MySQL 8.0 with the InnoDB engine

Our data is relational

Users, organizations, items, reservations, exchanges and reports are tightly linked. Foreign keys keep every link valid.

Transactions and row locks

Two students can never reserve the same item at the same moment.

Rules enforced inside the database

CHECK constraints, unique indexes, triggers, a stored procedure and a scheduled event.

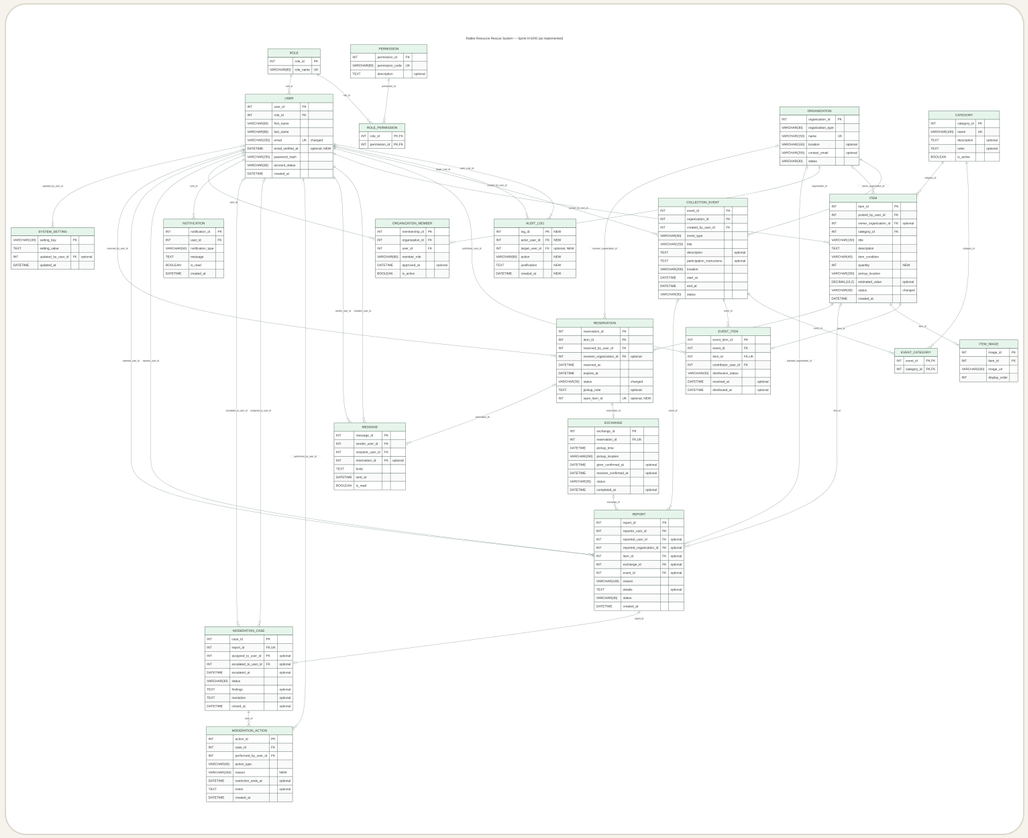
Fits our stack

Django’s ORM supports MySQL for Sprint IV, and managed MySQL hosting is widely available.



2 · UPDATED ER DIAGRAM

The ERD, generated from the live database



21 tables

137 fields

38 foreign keys

A script reads MySQL’s own catalog to draw this ERD and write the data dictionary, so both always match the real schema.

Crow’s foot: **||** exactly one, **|o** zero or one, **o{** zero or many. Fields are tagged PK, FK, UK, optional, NEW or changed.

Zoomable: ERD/Sprint-III-ERD.pdf · four area diagrams in the same folder

2 · WHAT CHANGED SINCE SPRINT II

Sprint II’s open questions, resolved

Change	Why
user.email_verified_at + @famu.edu CHECK	Verified campus email required to reserve
item.quantity	Giveaway listings must state a quantity
Item status **Pending Review**	Moderator “Pending Listings” screen
Reservation status **Pending**	Department “Approve Requests” screen
reservation.open_item_id + unique index	One open reservation per item, enforced by the database
moderation_action.reason (required)	Every moderation action records a reason
New table audit_log	Admin account actions must log a justification

Left out of scope: Student Org “Bulk Requests”. The requirements don’t call for item requests.

3 · IMPLEMENTED STRUCTURE

21 tables in six areas

Users & access

`user · role · permission · role_permission`

Organizations

`organization · organization_member`

Listings & exchanges

`category · item · item_image · reservation · exchange · message`

Collection drives

`collection_event · event_category · event_item`

Moderation

`report · moderation_case · moderation_action`

System

`notification · system_setting · audit_log`

38 foreign keys

27 CHECK rules

11 unique keys

8 triggers

2 + 1 + 1 views, procedure, event



4 · MEANINGFUL SAMPLE DATA

Realistic FAMU campus data for every role

Role	Users	Example
Campus User	7	Jordan Davis
Department Rep	3	Robert Hayes
Student Org Rep	3	Jasmine Carter
Community Moderator	2	Keisha Williams
Administrator	1	Angela Brooks

Sample listing	Owner	Status
Dell OptiPlex Desktops (×4)	CIS Department	Completed
TI-84 Plus Calculator	Elijah Moore	Reserved
Compound Microscope	Biology Dept.	Available
Reusable Water Bottles (×30)	Environmental Club	Available
Borosilicate Beaker Set	Biology Dept.	Pending Review

16 users · 3 departments and 3 student orgs · 26 items in every status · 10 reservations · 5 exchanges · 8 messages · 2 collection drives · 4 reports · 2 moderation cases · an audit-log entry

Four SQL scripts rebuild everything

```
# from Sprint Cycles/Sprint Cycle 3/Database
mysql -u root -p < 01_schema.sql
mysql -u root -p < 02_sample_data.sql
mysql -u root -p -t < 03_queries.sql
mysql -u root -p -t < 04_rule_tests.sql
# → 22 of 22 rule tests passed
```

Script	Does
01_schema	Tables, keys, constraints, views, triggers, procedure, event
02_sample_data	Loads the sample data
03_queries	Search, related, add, modify, reports
04_rule_tests	Tries to break each rule, then rolls back

MySQL features used: CREATE TABLE with FOREIGN KEY, CHECK, UNIQUE and FULLTEXT · generated columns · views · triggers with SIGNAL · stored procedures with transactions and FOR UPDATE locks · scheduled events · information_schema.



6 · RETRIEVE / SEARCH

Find available items in a category

```
SELECT title, item_condition, quantity, pickup_location, owner_name, owner_type
FROM v_available_items
WHERE category = 'Dorm Essentials' ORDER BY created_at DESC;
```

title	condition	qty	pickup_location	owner (type)
Twin XL Bedding Set	Like New	1	Gibbs Hall drop-off table	FAMU Environmental Club (Student Org)
700W Microwave	Good	1	Paddyfoote Complex desk	Jordan Davis (Individual)
Reusable Water Bottles	New	30	Student Union, Room 214	FAMU Environmental Club (Student Org)
LED Desk Lamp with USB Port	Like New	1	Paddyfoote Complex desk	Jordan Davis (Individual)

03_queries.sql, query A2 · also A1 user search, A3 full-text keyword search, A5 listings by creator



7 · RELATED DATA

A department’s items and who reserved them

```
SELECT i.title, i.quantity, i.status,
CONCAT(ru.first_name,' ',ru.last_name) AS reserved_by, ro.name AS reserved_for
FROM item i JOIN organization o ON o.organization_id = i.owner_organization_id
LEFT JOIN reservation r ON r.item_id = i.item_id AND r.status IN ('Pending','Active')
LEFT JOIN user ru ON ru.user_id = r.reserved_by_user_id
LEFT JOIN organization ro ON ro.organization_id = r.receiver_organization_id
WHERE o.name = 'Department of Computer & Information Sciences';
```

title	qty	status	reserved_by	reserved_for
Dell OptiPlex 7070 Desktops	4	Completed	NULL	NULL
24-inch Dell Monitors	6	Reserved	Tanya Mitchell	Department of Biology
Ergonomic Office Chairs	5	Available	NULL	NULL

03_queries.sql, query B2 · also B1 reservations by user, B4 drive donations, B6 moderation cases



8 · ADD DATA

List an item, then reserve it

```
-- 1. Imani lists a chair (new listings start as Pending Review)
INSERT INTO item (posted_by_user_id, category_id, title, description,
item_condition, pickup_location, estimated_value)
VALUES (11, 3, 'Folding Desk Chair', 'Padded folding chair…', 'Good', 'Student Union food court', 15.00);
-- 2. A moderator approves it; 3. Christopher reserves it in one transaction
UPDATE item SET status = 'Available' WHERE item_id = @new_item_id;
CALL reserve_item(@new_item_id, 14, NULL, 'Can pick up after 5pm.');
```

Step	item.status	What the database did
After INSERT	Pending Review	Default status; item_id 27 assigned by AUTO_INCREMENT
After UPDATE	Available	Now appears in v_available_items
After CALL	Reserved	Reservation 11 Active; trigger set expires_at = +48 h; owner notified



9 · MODIFY DATA

One UPDATE completes an exchange

```
-- Brianna (giver) already confirmed the mini-fridge handover. Imani (receiver) confirms:
UPDATE exchange SET receiver_confirmed_at = NOW() WHERE exchange_id = 5;
```

Record	Before	After	Changed by
exchange 5	Scheduled	**Completed**	Trigger (both confirmed)
reservation 7	Active	**Completed**	Trigger on exchange
item 4 (Mini-Fridge)	Reserved	**Completed**	Trigger on reservation

Also in 03_queries.sql: edit a listing (D1) · cancel a reservation, which frees the item (D3) · change a setting (D4) · reinstate an account with a logged justification (D5).

03_queries.sql, query D2 shows the before and after rows



10 · INSTRUCTOR QUERY

Your turn: ask us a question about the data

```
-- Where we start
SHOW TABLES;
DESCRIBE reservation;
SELECT * FROM v_available_items;
SELECT * FROM v_reservation_details;
```

Key joins

item → user, organization, category

reservation → item, user, organization

exchange → reservation

organization_member: user ⇄ organization

Cheat sheet with ready-made patterns: Sprint Cycle 3/README.md



A Campus User reserves an item

Tables

user → reservation ← item, plus category, system_setting and notification

Relationships

A user makes many reservations (1:M). An item has many over time but only one open at a time.

Data stored

Who and which item, optional receiving organization, reserved_at, expires_at, status, pickup note

How the database makes it work

reserve_item locks the item. Triggers check the rules, set a 48 h expiry and mark it Reserved. An event expires no-shows.



Giver and receiver complete an exchange

Tables

reservation → exchange, plus message for arranging the pickup

Relationships

One-to-one: exchange.reservation_id is a unique foreign key. Messages optionally link to the reservation.

Data stored

pickup_time, pickup_location, giver_confirmed_at, receiver_confirmed_at, status, completed_at

How the database makes it work

A CHECK blocks Completed without both confirmations. Triggers cascade completion to the reservation and item.



A student organization runs a collection drive

Tables

organization → collection_event → event_item ← item; event_category → category

Relationships

An org hosts many events (1:M). Two associative tables resolve event ⇄ item and event ⇄ category.

Data stored

Title, dates, location, instructions, accepted item types, each donation’s contributor and distribution status

How the database makes it work

A trigger allows only approved student orgs to host. A CHECK keeps end after start. Donations go Pending → Distributed.

03_queries.sql, queries B4 and B5 show the Fall Move-Out Collection Drive



We tried to break the rules. The database said no.

Rule	Enforced by	MySQL’s response
Email unique and @famu.edu	UNIQUE + CHECK	chk_user_campus_email violated
One open reservation per item	Unique index	Duplicate entry for key uq_one_open…
Anti-hoarding: one per category	Trigger	“only one open personal reservation per category”
Exchange needs both confirmations	CHECK	chk_exchange_both_confirmed violated
Every item has a valid category	FOREIGN KEY	foreign key constraint fails
Admin account changes logged	Trigger + audit_log	“need @actor_user_id and @justification”

22/22

rule tests pass in 04_rule_tests.sql

Run live: mysql -u root -p -t < 04_rule_tests.sql · all test changes are rolled back



Everything is in the repository

```
Sprint Cycles/Sprint Cycle 3/
├─ Database/
│  ├─ 01_schema.sql
│  ├─ 02_sample_data.sql
│  ├─ 03_queries.sql
│  ├─ 04_rule_tests.sql
│  └─ tools/generate_docs.py
├─ ERD/ (PDF, PNG, Mermaid source)
├─ Data Dictionary/ (Excel, Markdown)
└─ README.md (setup + recreation)
```

Member	Sprint III role
Isaiah Miller	Scrum Master; use cases and business rules
Lloyd Lewis	Lead database engineer: schema, triggers, sample data
Naser Halloum	Updated ERD
Ondrea Wagner	Data dictionary; presentation

github.com/LewisLloyd96/Rattler-Resource-Rescue-System

Meeting documentation: Sprint III meeting notes in the repository



NEXT · SPRINT CYCLE IV



Authentication, authorization and data retrieval

Connect

Django models over the existing tables.

Secure

Sign-in, and role checks from role_permission.

Retrieve

Real data on the dashboard and search screens.

Questions?